

System Design Revision Roadmap

Complete Guide to System Architecture & Design Patterns

TABLE OF CONTENTS

Part 1: Functional Requirements

- CDN (Content Delivery Networks)
- Database (NoSQL, SQL, Sharding, Partitioning, Replication)
- Monolithic vs Microservice Architecture
- API Design & Gateway
- Messaging Queues
- Security
- Protocols
- Caching
- Storage
- Monitoring & Logging

Part 2: Non-Functional Requirements

- Scalability
- Availability & Reliability
- Consistency
- Performance
- CAP Theorem
- Cost Optimizations

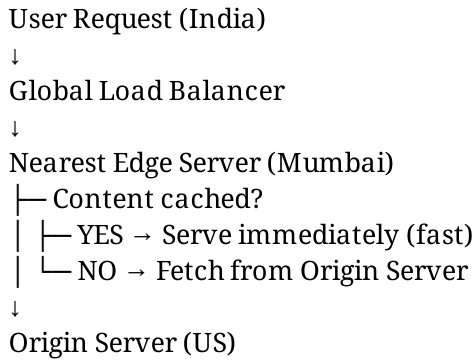
PART 1: FUNCTIONAL REQUIREMENTS

1.1 CDN (Content Delivery Networks)

Definition

A **Content Delivery Network (CDN)** is a geographically distributed network of servers strategically placed around the world to deliver web content (images, videos, stylesheets, scripts) to users from servers closest to their physical location, minimizing latency and improving load times.

Architecture Overview



Key Components

1. **Origin Server:** Primary server storing original content
2. **Edge Servers (PoP - Points of Presence):** Distributed globally, cache and serve content
3. **DNS Resolution:** Routes users to nearest edge server
4. **Content Distribution Nodes:** Manage routing and optimization
5. **Control Plane:** Manages caching policies, routing, load balancing

CDN Topologies

1. **Flat Topology:** All edge servers directly connected to origin (works for small CDNs)
2. **Hierarchical Topology:** Multi-tier structure (reduces load on origin server)
3. **Mesh Topology:** Edge servers interconnected (improves redundancy)
4. **Hybrid Topology:** Combines multiple topologies for optimal performance

Routing Strategies

1. **DNS-based Routing:** DNS response returns IP of nearest edge server
2. **GeoIP-based Routing:** Route based on user's geographic location
3. **Anycast Routing:** Multiple servers with same IP, request routed to nearest

Caching Strategies

1. **Push CDN:** Origin server proactively pushes content to edge servers (good for static content)
2. **Pull CDN:** Edge servers fetch content on demand (good for dynamic content)

When to Use CDN

- Serving static assets globally (images, CSS, JS, fonts)
- Reducing origin server load
- Improving availability during traffic spikes
- Reducing latency for worldwide users

CDN Performance Benefits

- **Reduced Latency:** Content from nearby server
 - **Faster Load Times:** Parallel downloads from multiple locations
 - **Better Availability:** Distributed architecture provides redundancy
 - **DDoS Protection:** Scrubber servers filter malicious traffic
 - **Bandwidth Savings:** Origin server traffic reduced
-

1.2 Database Systems

1.2.1 NoSQL (Not Only SQL)

Definition

NoSQL databases are non-relational databases designed for distributed, scalable data storage with flexible schemas. They prioritize availability and partition tolerance over strict consistency.

Characteristics

- Flexible schemas (schema-less)
- Distributed architecture by design
- High scalability and availability
- Eventual consistency
- Fast writes and reads at scale

Types

1. **Key-Value:** Redis, Memcached, DynamoDB
2. **Document:** MongoDB, CouchDB
3. **Column-Family:** Cassandra, HBase
4. **Graph:** Neo4j

When to Use

- High-velocity data ingestion
 - Semi-structured data
 - Rapid schema evolution
 - High scalability needed
 - Geographic distribution required
-

1.2.2 SQL (Relational Databases)

Definition

SQL (Relational) Databases store data in structured tables with predefined schemas, relationships, and enforce data integrity through ACID properties.

Characteristics

- Structured schema
- ACID compliance
- Strong consistency
- Normalized data structure
- Transaction support

Popular Options

PostgreSQL, MySQL, Oracle, SQL Server

When to Use

- Financial transactions
- Structured data with relationships
- Data integrity critical
- Complex queries with JOINS
- Regulatory compliance requirements

1.2.2.1 ACID Properties

ACID ensures reliable database transactions:

1. **Atomicity**: Transaction either completes fully or not at all (all-or-nothing)

Transfer \$100 from Account A to B:

- Debit from A: ✓
- Credit to B: ✗

Result: ROLLBACK (both operations undone)

2. **Consistency**: Database moves from one valid state to another

Before: A=\$1000, B=\$500, Total=\$1500

After: A=\$900, B=\$600, Total=\$1500

(Invariant: Total always \$1500)

3. **Isolation**: Concurrent transactions don't interfere with each other

Transaction 1: Read A

Transaction 2: Read A (sees same value, not T1's uncommitted changes)

4. **Durability**: Committed data persists despite failures

After COMMIT, data survives: power loss, crashes, disk failures

1.2.2.2 Database Transactions

Definition

A **transaction** is a sequence of operations treated as a single unit. Either all succeed or all fail.

Transaction Lifecycle

BEGIN

↓

EXECUTE Queries (SELECT, INSERT, UPDATE, DELETE)

↓

COMMIT (Success) OR ROLLBACK (Failure)

Isolation Levels (from weak to strong)

1. **Read Uncommitted:** Dirty reads possible
2. **Read Committed:** No dirty reads (default in most DBs)
3. **Repeatable Read:** Phantom reads possible
4. **Serializable:** Strictest (like sequential execution)

Common Issues & Solutions

- **Dirty Read:** Transaction reads uncommitted data from another
 - **Non-repeatable Read:** Same query returns different values within transaction
 - **Phantom Read:** New rows appear/disappear during transaction
-

1.2.3 In-Memory Databases

Definition

In-Memory Databases store data in RAM for ultra-fast access, used primarily for caching and temporary storage.

Examples

- Redis: Key-value store with data structures
- Memcached: Simple key-value cache
- Ignite: Distributed in-memory platform

Characteristics

- Sub-millisecond latency
- Volatile (data lost on restart)
- Limited by RAM size
- High throughput

When to Use

- Session storage
 - Real-time analytics
 - Caching frequently accessed data
 - Rate limiting counters
 - Leaderboards (sorted sets)
-

1.2.4 Cassandra Database

Definition

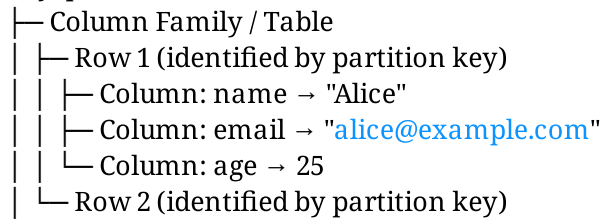
Cassandra is a wide-column, distributed NoSQL database designed for massive scalability and high availability across geographically distributed data centers.

Key Characteristics

- **Distributed:** No single point of failure
- **Fault-tolerant:** Data replicated across multiple nodes
- **Scalable:** Linear scalability with added nodes
- **Eventually Consistent:** Eventual consistency model
- **Column-oriented:** Stores data in columns not rows

Data Model

Keyspace (Database)



When to Use

- Time-series data (metrics, logs)
- Extremely high write throughput
- Geographic distribution critical
- Data center replication needed
- Not requiring complex transactions

1.2.5 Database Sharding

Definition

Sharding is a horizontal partitioning technique that distributes data across multiple database instances (shards) based on a shard key, enabling linear scalability.

Problem It Solves

Single Database (Bottleneck)

All queries → Single machine

vs.

Sharded Databases (Scalable)

User 1-1M queries → Shard 1

User 1M-2M queries → Shard 2

User 2M-3M queries → Shard 3

1.2.5.1 Horizontal Sharding

Data distributed across shards by rows

Database Shard 1: Users 0-999,999

Database Shard 2: Users 1,000,000-1,999,999

Database Shard 3: Users 2,000,000-2,999,999

Query: Get User 1,500,000 → Route to Shard 2

1.2.5.2 Vertical Sharding

Data distributed by columns across shards

Shard 1 (User Service): user_id, username, email

Shard 2 (Profile Service): user_id, bio, avatar, settings

Shard 3 (Payment Service): user_id, card_info, balance

Query User Profile: Fetch from Shard 1 + Shard 2

1.2.5.3 Key-Based Sharding (Hash-Based)

Shard determined by hashing shard key

$\text{shard_id} = \text{hash}(\text{user_id}) \% \text{num_shards}$

User 42: $\text{hash}(42) \% 3 = \text{Shard } 0$

User 99: $\text{hash}(99) \% 3 = \text{Shard } 1$

User 17: $\text{hash}(17) \% 3 = \text{Shard } 2$

Problem: Adding shards requires rehashing all data (maintenance intensive)

1.2.5.4 Directory-Based Sharding

Lookup table maintains shard mapping

Shard Directory (Lookup Table)

Shard ID	Key Range
Shard 1	0-999
Shard 2	1000-1999
Shard 3	2000-2999

Query User 1500:

1. Check Directory: 1500 in range 1000-1999 → Shard 2
2. Query Shard 2

Benefits: Easy to add shards, rebalance data

Drawback: Lookup table is single point of failure

1.2.6 Database Partitioning

Definition

Partitioning (similar to sharding) divides data into smaller subsets for improved performance, availability, and manageability. Can be applied to single DB (logical) or multiple DBs (physical/sharding).

1.2.6.1 Horizontal Partitioning

Divide rows across partitions

Table: Orders (10M rows)

Partition 1: Orders created in Jan-Mar 2024 (2.5M rows)

Partition 2: Orders created in Apr-Jun 2024 (2.5M rows)

Partition 3: Orders created in Jul-Sep 2024 (2.5M rows)

Partition 4: Orders created in Oct-Dec 2024 (2.5M rows)

Query: Orders in Feb 2024 → Scan only Partition 1 (faster)

Benefits: Faster queries, easier maintenance, parallel processing

1.2.6.2 Vertical Partitioning

Divide columns across partitions

Orders Table (100 columns)

Partition 1 (Hot Data): order_id, customer_id, amount, status
(frequently accessed, in fast SSD)

Partition 2 (Cold Data): description, notes, metadata, history
(rarely accessed, in slower storage)

Benefits: Improved cache hit ratio, better performance for hot data

1.2.6.3 Consistent Hashing

Algorithm for distributed data placement without full rehashing

Problem: Regular Hash Sharding

$\text{shard_id} = \text{hash}(\text{key}) \% \text{num_shards}$

Current: 3 shards

Key 42: $\text{hash}(42) \% 3 = 1 \rightarrow \text{Shard 1}$

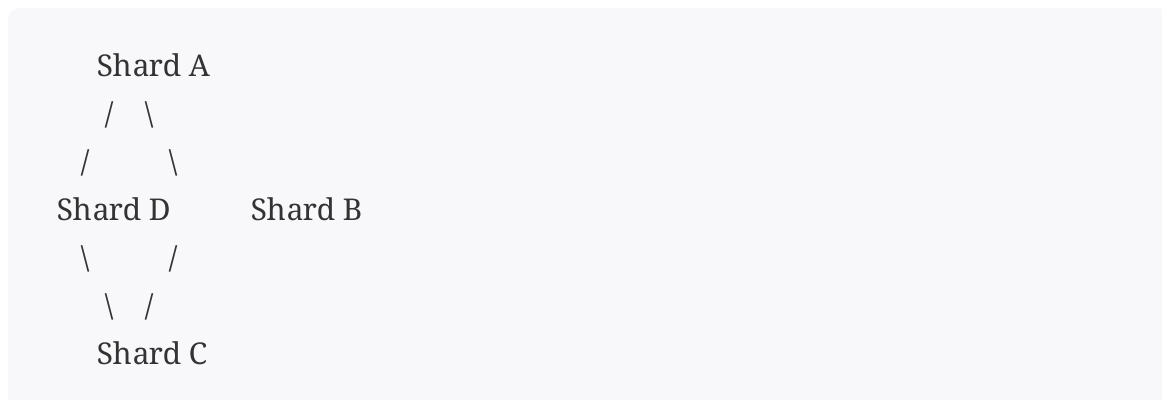
Add new shard (now 4 shards):

Key 42: $\text{hash}(42) \% 4 = 2 \rightarrow \text{Shard 2 (MOVED! Requires migration)}$

Result: ~67% of data must be moved (expensive)

Solution: Consistent Hashing

Hash Ring (0 to 2^{32})



Key 42: $\text{Hash}(42) = 573,892,934$

→ Find clockwise node → Shard B

Add Shard E:

Insert between C and D

Only keys between C and E move (25% instead of 67%)

Remove Shard C:

Only C's keys redistribute (25% instead of 67%)

Benefits

- Minimal data movement on node addition/removal
- Even distribution of keys
- Supports replication naturally (next N nodes on ring)

Implementation Details

1. Hash both keys and servers on same ring
2. To find server for key: hash(key), move clockwise until finding server
3. Replication: Store on N consecutive servers clockwise
4. Node failure: Keys redistribute to next nodes automatically
5. New node: Takes keys from next clockwise node

1.2.7 Database Replication

Definition

Replication is copying data from one database (master) to others (replicas) to improve availability, fault tolerance, and read performance.

Replication Architecture

Master (Write-only)

↓ (Replication stream)

├─ Replica 1 (Read)

├─ Replica 2 (Read)

└─ Replica 3 (Read)

Applications:

Writes → Master

Reads → Any Replica (load balanced)

1.2.7.1 Synchronous Replication

Master waits for replicas to acknowledge before confirming write

Client → Master: Write "Alice"

Master → Replica 1: Replicate

Master → Replica 2: Replicate

Replicas acknowledge ✓

Master acknowledges to Client ✓

Pros: Strong consistency, all replicas up-to-date

Cons: High latency (waits for slowest replica), throughput reduced

1.2.7.2 Asynchronous Replication

Master confirms write immediately, replicates in background

Client → Master: Write "Alice"

Master acknowledges immediately ✓

Master → Replica 1: Replicate (background)

Master → Replica 2: Replicate (background)

Pros: Low latency, high throughput

Cons: Weak consistency (replicas lag), data loss possible on master failure

1.2.7.3 Semi-Synchronous Replication

Master waits for at least one replica, others async

Client → Master: Write "Alice"

Master → Replica 1 (Sync): Wait for acknowledgment

Master → Replica 2 (Async): Send in background

When Replica 1 acks: Master acks to client ✓

Pros: Balance of consistency and performance

Cons: More complex, fallback to async on replica failure

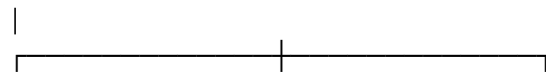
1.2.7.4 Master-Slave Architecture

Single master for writes, multiple slaves for reads

Architecture:

Master

(Writes)



↓↓↓

Slave 1 Slave 2 Slave 3

(Reads) (Reads) (Reads)

Read Load: 4x capacity (1 master, 3 slaves)

Failure Scenarios:

✗ Slave fails: No impact, others continue serving reads

✗ Master fails: Promote one slave to master (application-dependent)

When to Use Each Replication Type

- **Sync:** Financial systems, where consistency is critical
 - **Async:** Logging, analytics, user activity (eventual consistency acceptable)
 - **Semi-Sync:** Cache systems, balance needed
-

1.3 Monolithic vs Microservice Architecture

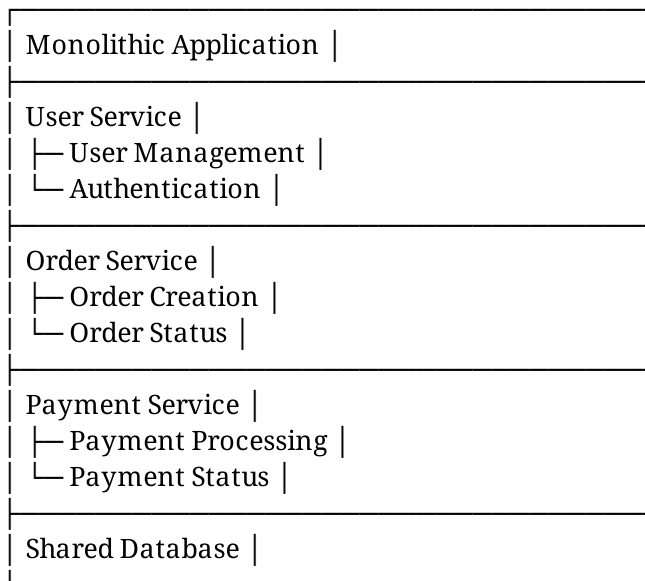
Monolithic Architecture

Definition

Monolithic architecture is a single, unified codebase deployed as one unit, where all components (UI, business logic, data access) are tightly coupled.

Structure

Single Codebase / Binary / Process



Deployment: Entire application as one unit

Advantages

1. **Simple to Develop:** Single codebase, easier IDE support
2. **Fast Deployment:** Deploy entire app once
3. **Easy Testing:** End-to-end testing is straightforward
4. **Low Latency:** No network calls between components
5. **Reduced Operational Complexity:** Single deployment unit

Disadvantages

1. **Scalability Limited:** Scale entire app, not individual services
2. **Technology Lock-in:** Choose technology for entire app
3. **Deployment Risk:** Bug in one component requires entire app redeploy
4. **Team Bottleneck:** Single codebase = coordination overhead
5. **Difficult to Maintain:** Large codebase becomes hard to understand

When to Use

- Small teams (< 5 developers)
- Simple domain logic
- MVP/Startup phase
- Tight latency requirements
- Not expecting massive scale

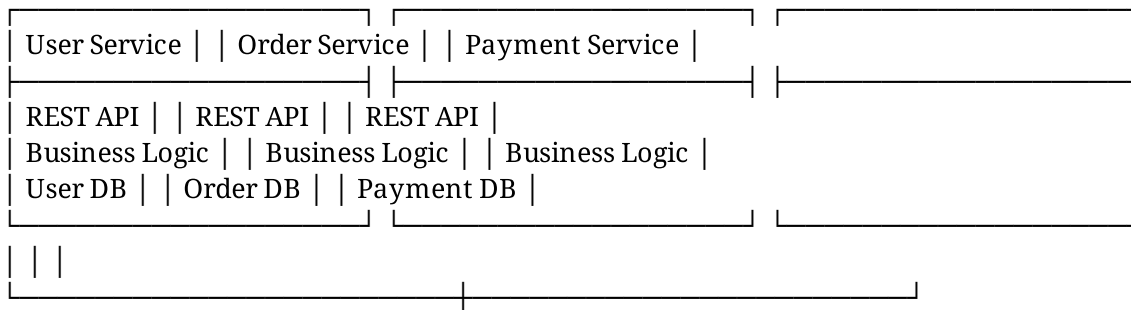
Microservice Architecture

Definition

Microservices architecture decomposes application into small, independent services, each running in separate processes, communicating via APIs.

Structure

Independent Services (Loosely Coupled)



API Gateway / Service Registry
(Service Discovery, Routing)

Deployment: Each service independently deployed

Communication: Synchronous (HTTP/REST) or Asynchronous (Message Queue)

Advantages

1. **Independent Scalability:** Scale only services that need it
2. **Technology Flexibility:** Each service uses best technology
3. **Fault Isolation:** Failure in one service doesn't crash others
4. **Faster Deployments:** Deploy service without affecting others
5. **Team Autonomy:** Each team owns one service
6. **Easy to Replace:** Update/replace individual service

Disadvantages

1. **Operational Complexity:** Multiple deployments, multiple technologies
2. **Network Latency:** Service-to-service calls over network
3. **Data Consistency:** Distributed transactions harder
4. **Debugging Difficult:** Trace issue across multiple services
5. **Requires DevOps Skills:** Containerization, orchestration needed

When to Use

- Large teams (> 15 developers)
- Complex domain (multiple teams, different specialties)
- Massive scale expected
- Different services have different technology needs
- Rapid deployment cycles required

Comparison Table

Aspect	Monolithic	Microservices
Deployment	Single unit	Independent
Scalability	Full app	Per-service
Technology	Single stack	Heterogeneous
Fault Tolerance	Risky	Isolated
Development	Simple	Complex
Team Size	Small	Large
Latency	Low	Higher (network)
Data Consistency	Easy (one DB)	Harder

1.4 API Design & Gateway

1.4.1 API Designing

Definition

An **Application Programming Interface (API)** is a contract defining how services communicate, specifying endpoints, request formats, response formats, and error handling.

1.4.1.1 REST (Representational State Transfer)

Principles

1. **Resource-oriented:** Everything is a resource (users, orders, products)
2. **Standard HTTP Methods:** GET, POST, PUT, DELETE, PATCH
3. **Status Codes:** Standardized responses (200, 201, 400, 404, 500)
4. **Stateless:** Each request has all info needed

Examples

GET /users → Fetch all users
GET /users/123 → Fetch user with ID 123
POST /users → Create new user
PUT /users/123 → Update user 123
DELETE /users/123 → Delete user 123
GET /users/123/orders → Fetch user 123's orders

HTTP Status Codes

- **2xx (Success):** 200 OK, 201 Created, 204 No Content
- **3xx (Redirect):** 301 Moved Permanently, 302 Found
- **4xx (Client Error):** 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
- **5xx (Server Error):** 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Advantages

- Simple and intuitive
- Cacheable (GET requests)
- Uses standard HTTP
- Scalable and stateless

Disadvantages

- Over-fetching (get more data than needed)
- Under-fetching (need multiple requests)
- No real-time updates (polling needed)

1.4.1.2 SOAP (Simple Object Access Protocol)

Overview

SOAP is an older, more formal protocol using XML for message format, often with WSDL (Web Services Description Language).

SOAP Request (XML-based):

`soap:Envelope`

`soap:Body`

123

`</soap:Body>`

`</soap:Envelope>`

Differences from REST

Aspect	REST	SOAP	
Message Format	JSON	XML	
Protocol	HTTP	HTTP/SMTP	
Complexity	Simple	Complex	
Performance	Fast	Slower	
Caching	Yes	No	
Industry Adoption	Modern	Legacy	

When to Use

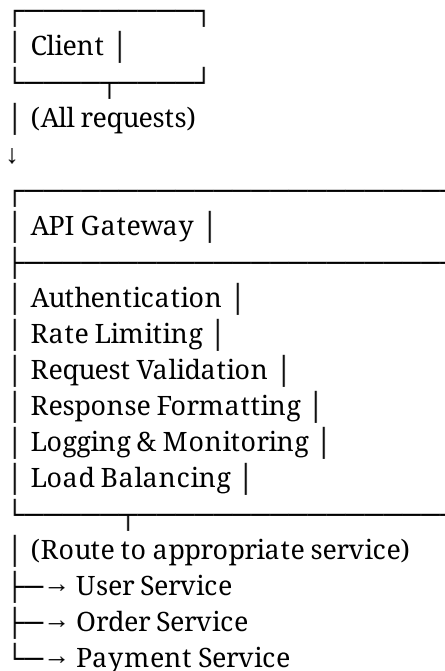
- SOAP: Enterprise systems, strict contracts required
- REST: Modern web APIs, microservices, mobile apps

1.4.2 API Gateway

Definition

An **API Gateway** is a server that acts as intermediary between clients and backend services, handling routing, authentication, rate limiting, and other cross-cutting concerns.

Architecture



Benefits

1. **Single Entry Point:** All traffic through one gateway
2. **Cross-cutting Concerns:** Centralized auth, logging, monitoring
3. **Request Transformation:** Modify requests before forwarding
4. **Load Balancing:** Distribute across backend services
5. **Rate Limiting:** Prevent abuse
6. **API Versioning:** Support multiple API versions

1.4.2.1 Load Balancing

Definition

Load Balancing distributes incoming requests across multiple servers to prevent any single server from becoming a bottleneck.

Load Balancing Algorithms

1. **Round Robin:** Requests to servers in circular order
Request 1 → Server 1
Request 2 → Server 2
Request 3 → Server 3
Request 4 → Server 1 (cycle repeats)
2. **Least Connections:** Route to server with fewest active connections
Server 1: 10 active connections
Server 2: 5 active connections ← Route here
Server 3: 15 active connections
3. **Weighted Round Robin:** Servers with different capacities get proportional load
Server 1 (weight 2): 2 requests
Server 2 (weight 1): 1 request
4. **IP Hash:** Hash client IP to ensure consistent routing
Client A → Always Server 1 (session affinity)
Client B → Always Server 2
5. **Latency-based:** Route to server with lowest latency

When to Use

- Multiple backend instances
- Different server capacities
- Session affinity needed (IP hash)
- Geographic distribution (latency-based)

1.4.2.2 Rate Limiting

Definition

Rate Limiting restricts the number of requests a client can make within a time window, preventing abuse and ensuring fair resource usage.

Algorithms

Token Bucket Algorithm (Most Common)

Bucket Capacity: 100 tokens

Refill Rate: 10 tokens/second

Time 0s:

- └─ Bucket: 100 tokens
- └─ Request: Use 1 token ✓ (99 tokens left)
- └─ Request: Use 1 token ✓ (98 tokens left)

Time 1s:

- └─ Refill: +10 tokens (108 → capped at 100)
- └─ Request: Use 1 token ✓
- └─ Request: Use 100 tokens ✓ (burst!)

When bucket empty:

- └─ Request: 429 Too Many Requests ✗

Leaky Bucket Algorithm

Similar to token bucket, but requests are processed at fixed rate.
Prevents burst entirely, smoother output rate.

Rate Limiting Types

1. **Fixed Rate:** Same limit for all users
10 requests per second per user
2. **Sliding Window:** Time window slides for fairness
Limit: 100 requests per minute
Evaluated at: 1:05 - 2:05, 1:06 - 2:06, etc.
3. **Adaptive Rate Limiting:** Adjust based on system load
Normal load: 1000 RPS
High load: 500 RPS (reduce to protect)

Implementation

Redis-based Rate Limiting:

key = "rate_limit:" + client_id + ":" + window

INCR key

EXPIRE key 60

IF key > threshold

REJECT (429)

ELSE

ALLOW

1.4.2.2.1 Throttling

Definition

Throttling is a specific type of rate limiting that delays/queues excess requests instead of rejecting them.

Rate Limiting: REJECT excess (429 Too Many Requests)

Throttling: QUEUE and process later

Example:

Request Rate: 10 RPS

Request Load: 50 RPS

Rate Limiting approach:

10 processed ✓

40 rejected ✗

Throttling approach:

10 processed immediately ✓

40 queued, processed as capacity available

When to Use

- Protecting backend from overload
 - Batch processing systems
 - Background jobs
 - Important requests shouldn't fail
-

1.4.2.2.2 DDoS Attacks

Definition

Distributed Denial of Service (DDoS) attacks overwhelm servers by sending massive traffic from many sources, rendering services unavailable.

Types of DDoS Attacks

1. **Volumetric Attacks:** Flood bandwidth
Normal traffic: 1 Gbps
DDoS attack: 100 Gbps
→ ISP bandwidth exhausted
2. **Protocol Attacks:** Exploit protocol weaknesses
TCP SYN Flood: Send many SYN packets without completing handshake
→ Server resources exhausted
3. **Application Layer Attacks:** Exploit application logic
HTTP Flood: Many slow HTTP requests
→ Application threads exhausted

DDoS Mitigation Strategies

1. **Rate Limiting:** Limit requests per IP/user
2. **Geo-blocking:** Block traffic from certain countries
3. **WAF (Web Application Firewall):** Filter malicious patterns
4. **Anycast Routing:** Distribute across multiple data centers
5. **CDN Protection:** Absorb attack traffic
6. **Behavioral Analysis:** Detect anomalies

API Gateway DDoS Protection

API Gateway receives 1000 RPS:

- └ Normal traffic (50 RPS) → Pass through
 - └ Suspicious (500 RPS from 5 IPs) → Rate limit
 - └ Clearly malicious (450 RPS from 1 IP) → Block immediately
-

1.4.2.3 Routing

Definition

Routing in API Gateway directs incoming requests to appropriate backend services based on URL path, headers, or other criteria.

Routing Strategies

1. Path-based Routing

/users/* → User Service
/orders/* → Order Service
/payments/* → Payment Service

2. Header-based Routing

X-API-Version: v1 → Service v1
X-API-Version: v2 → Service v2

3. Host-based Routing

[api.example.com](#) → API Service
[admin.example.com](#) → Admin Service

4. Method-based Routing

GET /resource → Read Service (replica)
POST /resource → Write Service (primary)

Implementation

Request: GET /orders/123 with X-Client-ID: premium

Gateway Logic:

IF X-Client-ID == "premium"

Route to Premium Order Service (lower latency, priority)

ELSE

Route to Standard Order Service

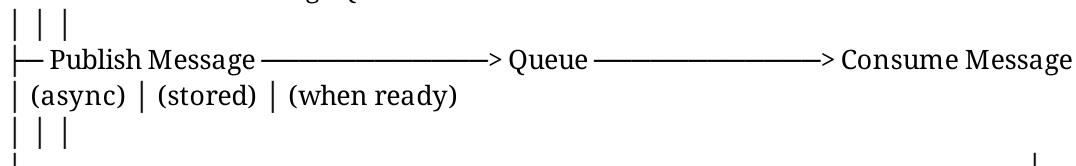
1.5 Messaging Queues

Definition

Message Queues are systems that enable asynchronous communication between services, allowing one service to send messages that another service processes later.

Architecture

Producer Service Message Queue Consumer Service



(Decoupled interaction)

Benefits

1. **Decoupling:** Services don't need to know about each other
 2. **Asynchronous:** Producer doesn't wait for consumer
 3. **Durability:** Messages persisted, survives failures
 4. **Scalability:** Consumer can process at own pace
 5. **Load Leveling:** Spikes buffered in queue
-

1.5.1 Kafka

Definition

Apache Kafka is a high-throughput, fault-tolerant distributed message broker designed for real-time data streaming.

Architecture

Topics: Logical channels for messages

Topic: "user_events"

└─ Partition 0: Messages 1-100

└─ Partition 1: Messages 101-200

└─ Partition 2: Messages 201-300

Producers: Write to topics

User Service → Publish to "user_events" topic

Consumers: Read from topics

Analytics Service → Subscribe to "user_events"

Notification Service → Subscribe to "user_events"

Key Characteristics

1. **Distributed:** Multiple brokers, data replicated
2. **High Throughput:** Millions of messages/second
3. **Persistence:** Messages retained for configurable period
4. **Consumer Groups:** Multiple consumers can read same topic (scales reading)
5. **Ordering:** Messages in partition ordered

Use Cases

- Real-time event streaming
- Log aggregation
- Metrics collection
- Activity tracking
- Stream processing (Kafka Streams)

Consumer Group Example

Topic: "orders" has 3 partitions

Consumer Group 1 (Analytics):

└─ Consumer A → Partition 0

└─ Consumer B → Partition 1

└─ Consumer C → Partition 2

Consumer Group 2 (Notifications):

└─ Consumer D → Partition 0

└─ Consumer E → Partition 1

└─ Consumer F → Partition 2

Each group independently tracks progress (offset)

1.5.2 RabbitMQ

Definition

RabbitMQ is a traditional message broker implementing Advanced Message Queuing Protocol (AMQP), designed for reliable message delivery.

Architecture

Producers → Exchange → Queues → Consumers

- |
- |— Bindings (routing rules)
- | If routing_key="ordercreated" → Queue "orders"
- | If routing_key="usercreated" → Queue "users"

Exchange Types

1. **Direct Exchange:** Route based on exact routing key match
Producers publish with routing_key="orderplaced"
Queue bound with binding_key="orderplaced"
→ Message routed to matching queue
2. **Topic Exchange:** Route based on pattern matching
Producers:
order* → order-related messages
user* → user-related messages
Queues:
*.placed → Receive all "placed" events
order# → Receive all order events
3. **Fanout Exchange:** Broadcast to all queues
One message → All bound queues
(Similar to pub/sub)

Key Characteristics

1. **Reliability:** Message persistence, acknowledgements
2. **AMQP Compliant:** Standard protocol
3. **Flexible Routing:** Exchanges with bindings
4. **Clustering:** Multiple nodes for HA
5. **Moderate Throughput:** Slower than Kafka

Use Cases

- Traditional job queues
- Microservice communication
- Email sending pipelines
- Task distribution

Kafka vs RabbitMQ

Aspect	Kafka	RabbitMQ
Throughput	Very High	Moderate
Message Retention	Long (days)	Short (hours)

Use Case	Event Streaming	Job Queues
Complexity	More complex	Simpler
Consumer Scaling	Partitions	Queue
Real-time Analytics	Yes	No
Learning Curve	Steep	Gentle

1.6 Security

1.6.1 OAuth

Definition

OAuth is an open authorization protocol allowing users to grant third-party applications access to their resources without sharing passwords.

Flow (OAuth 2.0)

User wants to login to App with Google account:

1. User clicks "Login with Google"
- ↓
2. App redirects to Google OAuth endpoint
- ↓
3. User grants permission to App
- ↓
4. Google redirects back with authorization code
- ↓
5. App exchanges code for access token
- ↓
6. App uses token to access user's Google data (email, profile)
- ↓
7. User logged into App

Key Concepts

- **Authorization Code:** Temporary code, exchanged for token
- **Access Token:** Allows access to protected resources
- **Refresh Token:** Obtain new access token when expired
- **Scope:** Permissions requested (email, profile, etc.)

When to Use

- Third-party login (Google, GitHub, Facebook)
- API access on behalf of user
- Mobile app authentication

1.6.2 Encryption

Definition

Encryption converts readable data (plaintext) to unreadable format (ciphertext) using mathematical algorithms and keys, only decryptable with correct key.

Types

1. **Symmetric Encryption:** Same key for encrypt/decrypt
Plaintext: "Hello World"
Key: "secret123"
Algorithm: AES-256
Encrypt: "Hello World" → "xK8#mP2qL9@" → "Hello World"
(Same key used for both)
Algorithms: AES, DES, 3DES
2. **Asymmetric Encryption:** Public key to encrypt, private key to decrypt
Public Key: Known to everyone (encrypt with this)
Private Key: Secret (only owner has, decrypt with this)
Anyone can encrypt → Only owner can decrypt
Algorithms: RSA, ECC
3. **Hashing:** One-way encryption, same input = same output
Password: "myPassword"
Hash: "5f4dcc3b5aa765d61d8327deb882cf99"
Cannot reverse-engineer password from hash
Used for password storage, integrity verification
Algorithms: MD5, SHA-1, SHA-256, bcrypt

When to Use

- **Symmetric:** Large data, need speed (database encryption)
- **Asymmetric:** Key exchange, digital signatures
- **Hashing:** Password storage, data integrity

HTTPS (TLS/SSL)

HTTP (Unencrypted): Browser ↔ Server (anyone can intercept)

HTTPS (TLS/SSL): Browser ↔ Encrypted ↔ Server (safe)

Handshake:

1. Client and server agree on encryption algorithm
 2. Server shares public certificate
 3. Client verifies certificate (trusted CA)
 4. Session key generated and encrypted
 5. All communication encrypted with session key
-

1.6.3 JWT (JSON Web Token)

Definition

JWT is a compact token format containing encoded claims (statements about identity), used for stateless authentication and authorization.

Structure

JWT:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

Three parts separated by dots:

1. Header (Base64): {"alg":"HS256","typ":"JWT"}
 - Algorithm: HS256 (HMAC SHA-256)
 - Type: JWT
2. Payload (Base64): {"sub":"123","name":"John Doe","iat":1516239022}
 - sub: Subject (user ID)
 - name: User name
 - iat: Issued at (timestamp)
 - exp: Expiration (optional)
3. Signature (Base64): HMAC(header:payload, secret)
 - Verifies token hasn't been tampered with
 - Server-only secret used to sign

JWT Flow

1. User logs in with credentials
↓
2. Server validates, creates JWT with user ID and permissions
↓
3. Server returns JWT to client
↓
4. Client stores JWT (localStorage, httpOnly cookie)
↓
5. Client sends JWT with each request (Authorization header)
↓
6. Server verifies signature, extracts claims, allows access
↓
7. No server-side session needed (stateless)

Advantages

1. **Stateless:** No server-side session storage
2. **Scalable:** Works across multiple servers
3. **Mobile-friendly:** Works with native apps
4. **Cross-domain:** Works across CORS boundaries
5. **Standard:** RFC 7519

Disadvantages

1. **Token Revocation:** Hard to revoke immediately (blacklist needed)
2. **Token Size:** Larger than session ID
3. **Exposed Claims:** Payload is Base64 (not encrypted), don't put secrets

When to Use

- Single Page Applications (SPAs)
- Mobile app authentication
- Microservice authentication
- API access tokens

1.7 Protocols

1.7.1 HTTP (HyperText Transfer Protocol)

Definition

HTTP is the foundation of web communication, defining how messages are formatted and transmitted between clients and servers.

Versions

1. **HTTP/1.1** (1997): Long-lived connections, still widely used
 - Persistent connections (keep-alive)
 - Pipelining (multiple requests before response)
 - Header compression via tools like gzip
2. **HTTP/2** (2015): Multiplexing for better performance
 - Multiplexing: Multiple requests/responses simultaneously on one connection
 - Server push: Server proactively sends resources
 - Binary format: More efficient parsing
 - Header compression: HPACK
3. **HTTP/3** (2022): QUIC-based, fastest
 - Faster connection setup (0-RTT)
 - Better mobile support (connection migration)
 - Multiplexing at transport layer

Request-Response Model

HTTP/1.1 GET /api/users/123 HTTP/1.1

Host: api.example.com

Authorization: Bearer token123

Accept: application/json

HTTP/1.1 200 OK

Content-Type: application/json

Content-Length: 56

{"id": 123, "name": "Alice", "email": "alice@example.com"}

Methods

- **GET**: Retrieve resource
 - **POST**: Create resource
 - **PUT**: Replace resource
 - **PATCH**: Partially update
 - **DELETE**: Remove resource
 - **HEAD**: Like GET but no response body
 - **OPTIONS**: Describe communication options
-

1.7.2 TCP-UDP

Definition

Transport layer protocols for sending data over networks. TCP is reliable, UDP is fast.

TCP (Transmission Control Protocol)

Reliable, ordered delivery

Connection Setup (3-way handshake):

Client → Server: SYN (I want to connect)

Server → Client: SYN-ACK (I agree)

Client → Server: ACK (Connection established)

Data Transfer:

Client → Server: Data packets

Server → Client: ACK (I received packet N)

If packet lost: Server requests retransmit

Connection Close:

Client → Server: FIN (I'm done)

Server → Client: ACK + FIN

Client → Server: ACK

UDP (User Datagram Protocol)

Fast, unreliable delivery

No handshake, just send:

Client → Server: Datagram 1

Client → Server: Datagram 2

Server → Client: Datagram 3

If packet lost: No retransmit, no acknowledgement

Comparison

Aspect	TCP	UDP	
Reliability	100%	Best effort	
Speed	Slower	Faster	
Ordering	Guaranteed	Not ordered	
Overhead	High	Low	

Connection	Required	Connectionless
Retransmit	Yes	No

Use TCP: Web, Email, File Transfer (need reliability)

Use UDP: Video, VoIP, Gaming, DNS (can tolerate loss)

1.7.3 WebSockets

Definition

WebSockets enable bidirectional, real-time communication between client and server over a single persistent connection, unlike HTTP's request-response model.

How It Works

1. Handshake (HTTP upgrade):

Client: GET /socket HTTP/1.1

Upgrade: websocket

Connection: Upgrade

Server: HTTP/1.1 101 Switching Protocols

Upgrade: websocket

2. Persistent Connection:

WebSocket Connection (bidirectional)

Client ← → Server (real-time messages)
--

3. Bidirectional Communication:

Server can push to client anytime

(unlike HTTP polling where client always asks)

HTTP Polling vs WebSocket

HTTP Polling (Traditional):

Client asks "Any updates?" every 1 second

Most responses: "No updates"

High latency, high bandwidth waste

WebSocket (Real-time):

Connection always open

Server pushes updates immediately

Low latency, efficient bandwidth

Use Cases

- Live notifications
- Chat applications
- Real-time dashboards
- Collaborative editing
- Live feeds

Example

JavaScript:

```
const ws = new WebSocket("ws://api.example.com/socket");
```

```
ws.onmessage = (event) => {  
  console.log("Received:", event.data); // Real-time message  
};
```

```
ws.send("Hello Server"); // Send to server
```

1.8 Caching

Definition

Caching stores frequently accessed data in fast-access storage (RAM) to reduce latency and load on slower primary storage (disk/database).

1.8.1 Redis & Memcached

Redis

In-memory data structure store

Features:

- └─ Key-value pairs
- └─ Strings, Lists, Sets, Sorted Sets, Hashes
- └─ Expiration (automatic eviction)
- └─ Atomic operations (INCR, APPEND, etc.)
- └─ Pub/Sub messaging
- └─ Persistence (optional)
- └─ Replication & clustering

Performance: ~100,000 operations/second

Data Types: Rich variety

Use Cases:

- Session storage
- Cache layer
- Rate limiting
- Leaderboards (sorted sets)
- Real-time counters
- Pub/Sub messaging

Memcached

Simple key-value cache

Features:

- └─ Key-value pairs only
- └─ Expiration
- └─ Distributed (multiple servers)

- └ No persistence
- └ Consistent hashing

Performance: ~1 million operations/second (faster)

Simplicity: Simpler than Redis

Use Cases:

- Database query caching
- Session caching
- HTML fragment caching

Comparison

Feature	Redis	Memcached
Data Types	Rich (7+)	String only
Persistence	Yes (RDB/AOF)	No
Performance	Good	Excellent
Memory Efficient	Less (features)	More (simple)
Replication	Yes	No
Pub/Sub	Yes	No
Complexity	More	Less

1.8.2 Hibernate Caching

Overview

Hibernate is an ORM (Object-Relational Mapping) framework for Java. It provides multi-level caching to reduce database queries.

1.8.2.1 L1 Cache (Session Cache)

Definition: Session-scoped cache

Scope: Single Hibernate Session

Lifetime: Session duration

Storage: Application memory

Example:

```
Session session = sessionFactory.openSession();
```

```
User user = session.get(Userclass, 1); // Query DB
```

```
User user2 = session.get(Userclass, 1); // From L1 cache (no DB hit!)
```

```
session.close(); // L1 cache destroyed
```

Behavior:

- └ Automatic (always enabled)
- └ Session-level isolation
- └ Garbage collected with session
- └ Not shared across sessions

1.8.2.2 L2 Cache (SessionFactory Cache)

Definition: SessionFactory-scoped cache

Scope: Entire application

Lifetime: Application lifetime

Storage: Distributed cache (Redis, Memcached, etc.)

Example:

```
User user = session1.get(User.class, 1); // Query DB, stored in L2
session1.close();
```

```
User user2 = session2.get(User.class, 1); // From L2 cache (no DB hit!)
session2.close();
```

Behavior:

- └ Optional (must enable)
- └ Shared across sessions
- └ Outlives sessions
- └ Distributed across servers
- └ Requires configuration

Providers:

- Redis
- Memcached
- EHCache
- Hazelcast

L1 vs L2 Cache

Query Sequence:

1. Check L1 Cache (current session) → MISS
2. Check L2 Cache (shared) → MISS
3. Query Database
4. Store in L1 Cache
5. Store in L2 Cache

Next query (same session):

1. Check L1 Cache → HIT (return immediately)

Query from different session:

1. Check L1 Cache (different session) → N/A (different cache)
2. Check L2 Cache → HIT (from other session's store)

Cache Invalidation Strategies

When data changes in DB:

1. Write-Through: Update cache when writing DB
Write → DB ✓ → Cache ✓

2. Write-Behind: Write cache immediately, DB later
Write → Cache ✓ → DB (async) ✓
3. Invalidation: Remove from cache on write
Write → DB ✓ → Invalidate Cache → Next read: DB query

1.9 Single Point of Failures

Definition

A **Single Point of Failure (SPOF)** is a component whose failure causes entire system to become unavailable.

Examples

Architecture with SPOF:

Web Servers
Load Balancer (SPOF ✖)
Database (SPOF ✖)

If load balancer fails: No requests routed

If database fails: All read/write operations fail

Eliminating SPOFs

1. **Redundancy:** Multiple instances of critical components
Database (Master) → Replicas (Slave 1, 2, 3)
Load Balancer 1 → Load Balancer 2 (standby)
2. **Failover:** Automatic recovery when component fails
Primary fails → Automatically use secondary
3. **Health Checks:** Detect and replace failed instances
Every 30s: Check if service alive
If failed: Remove from load balancer, start replacement

Resilience Patterns

1. **Circuit Breaker:** Stop calling failing service temporarily
Closed (normal): Call service
Open (failing): Don't call, return error immediately
Half-Open (testing): Try calling again, if works → close
2. **Retry with Exponential Backoff:** Retry failed requests with increasing delays
Attempt 1: Immediately
Attempt 2: Wait 1 second
Attempt 3: Wait 2 seconds
Attempt 4: Wait 4 seconds
3. **Bulkhead Pattern:** Isolate critical resources
Thread Pool 1 (High priority): 100 threads
Thread Pool 2 (Normal): 50 threads
If normal is overloaded, high priority still responsive

1.10 Monitoring & Logging

Definition

Monitoring tracks system health and performance (real-time metrics), while **Logging** records system events for debugging and analysis.

1.10.1 Health Checks

Ping Endpoint (Liveness Probe):

GET /health

Response: 200 OK (service alive)

Readiness Probe:

GET /ready

Response: 200 OK (ready to serve requests)

Load balancer uses these to:

- └─ Detect dead instances
- └─ Remove from rotation
- └─ Start replacements

1.10.2 Metrics

Quantifiable measurements of system behavior

Types:

1. Counter: Increases over time
 - └─ Total requests: 1,234,567
 - └─ Errors: 1,234
 - └─ Exceptions: 56
2. Gauge: Current value
 - └─ Active connections: 4,523
 - └─ Memory usage: 8.5 GB
 - └─ CPU usage: 45%
3. Histogram: Distribution of values
 - └─ Request latency: 10ms, 15ms, 20ms, 100ms, 500ms
 - └─ Response size: 1KB, 5KB, 10KB, 100KB
4. Summary: Aggregate statistics
 - └─ p50 (median): 50ms
 - └─ p95: 200ms
 - └─ p99: 1000ms
 - └─ Mean: 150ms

Commonly Tracked Metrics

- **Latency:** Response time (p50, p95, p99)
 - **Throughput:** Requests per second (RPS)
 - **Error Rate:** Failed requests percentage
 - **Resource Utilization:** CPU, Memory, Disk usage
 - **Queue Depth:** Messages waiting in queue
-

1.10.3 Alerting

Automated notifications when metrics exceed thresholds

Alert Configuration:

IF error_rate > 5% THEN send_alert
IF p99_latency > 1000ms THEN send_alert
IF cpu_usage > 80% THEN send_alert
IF disk_space < 10% THEN send_alert

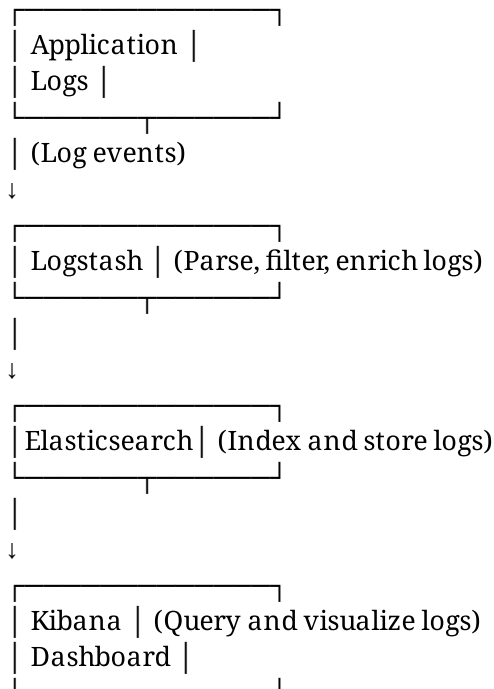
Actions:

- └ Email to team
 - └ Slack message
 - └ PagerDuty (on-call alert)
 - └ Auto-remediation (scale up, restart service)
-

1.10.4 ELK Stack

Elasticsearch, Logstash, Kibana - Centralized logging

Architecture:



Workflow:

1. App logs to Logstash
2. Logstash parses (extract timestamp, level, service)
3. Store in Elasticsearch (indexed)
4. Kibana queries and displays
5. Users search logs and create dashboards

Benefits

- **Centralized:** All logs in one place
 - **Searchable:** Full-text search across all logs
 - **Real-time:** Live log streaming
 - **Visualization:** Dashboards and graphs
 - **Alerting:** Alert on patterns
-

1.11 Storage

1.11.1 BLOB Storage

Binary Large Object - Unstructured data storage

Examples: Images, Videos, Documents, Archives

Characteristics:

- └─ Unstructured data
- └─ Large files (MB to GB)
- └─ Read-heavy (rarely updated)
- └─ No schema needed

Solutions:

- AWS S3
- Google Cloud Storage
- Azure Blob Storage
- MinIO (self-hosted)

Typical architecture:

- └─ Application: Read/Write via HTTP API
 - └─ Metadata: Database (filename, size, upload date)
 - └─ Actual file: BLOB storage
-

1.11.2 Relational Storage

Structured, schema-based data

Examples: User info, Orders, Transactions, Relationships

Characteristics:

- └─ Structured schema
- └─ Relationships between tables
- └─ ACID compliance
- └─ Complex queries (JOINS)
- └─ Normalized data

Solutions:

- PostgreSQL
- MySQL
- Oracle
- SQL Server

Typical architecture:

- └─ Multiple tables with relationships
 - └─ Indexes for fast queries
 - └─ Foreign keys for data integrity
 - └─ Views for abstraction
-

PART 2: NON-FUNCTIONAL REQUIREMENTS

2.1 Scalability

Definition

Scalability is system's ability to handle increased load (users, data, requests) without performance degradation.

2.1.1 Vertical Scaling (Scale Up)

Add more resources to single machine

Before:

Server 1: 4 CPU, 8 GB RAM

After:

Server 1: 16 CPU, 64 GB RAM (upgraded)

Pros:

- └─ Simple (no architecture changes)
- └─ No coordination needed
- └─ Lower latency (local memory)

Cons:

- └─ Limited by hardware (max CPU/RAM)
 - └─ Downtime during upgrade
 - └─ Single point of failure
 - └─ Expensive (diminishing returns)
-

2.1.2 Horizontal Scaling (Scale Out)

Add more machines to distributed system

Before:

Server 1: 4 CPU, 8 GB RAM

Server 2: 4 CPU, 8 GB RAM

After:

Server 1: 4 CPU, 8 GB RAM

Server 2: 4 CPU, 8 GB RAM

Server 3: 4 CPU, 8 GB RAM (new)

Server 4: 4 CPU, 8 GB RAM (new)

Load balancer distributes traffic:

- |— Request 1 → Server 1
- |— Request 2 → Server 2
- |— Request 3 → Server 3
- |— Request 4 → Server 4

Pros:

- |— No hardware limits
- |— Better fault tolerance (failure of one doesn't crash all)
- |— No downtime
- |— Cost-effective (commodity hardware)

Cons:

- |— Complex coordination
 - |— Network latency (cross-server calls)
 - |— Distributed transaction challenges
 - |— Operational complexity
-

2.1.3 Auto Scaling

Automatically add/remove resources based on demand

Metrics Monitored:

- |— CPU usage > 70% → Scale up
- |— CPU usage < 20% → Scale down
- |— Request queue depth > threshold → Scale up
- |— Response time > threshold → Scale up

Scaling Policies:

1. Scale Up: Add 2 servers
2. Wait 5 minutes (stability period)
3. If still high: Add 2 more

Down:

1. Remove 1 server
2. Wait 10 minutes
3. If still low: Remove another

Cloud Providers:

- AWS Auto Scaling Groups
 - Kubernetes (horizontal pod autoscaler)
 - Google Cloud Autoscaling
 - Azure Scale Sets
-

2.2 Availability & Reliability

2.2.1 Uptime

Percentage of time system is operational

Uptime SLAs (Service Level Agreements):

99.9% (3 nines):

- └ Annual downtime: ~8.7 hours
- └ Typical: Most web services target this
- └ Acceptable for: Social media, e-commerce

99.99% (4 nines):

- └ Annual downtime: ~52 minutes
- └ Typical: Financial systems, cloud infrastructure
- └ Requires: Redundancy, failover, monitoring

99.999% (5 nines):

- └ Annual downtime: ~5 minutes
- └ Typical: Telecommunications, power grids
- └ Extremely expensive
- └ Requires: Massive redundancy

Calculation:

If service down 10 hours in 1000 hours:

Uptime = $(1000 - 10) / 1000 = 99\%$

2.2.2 Fault Tolerance

Ability to continue operating despite component failures

With Redundancy:

User requests → LB (Load Balancer)

- └ Server 1 ✓
- └ Server 2 ✗ (failed)
- └ Server 3 ✓

LB detects Server 2 failed

Routes traffic to Server 1 & 3

System continues working (gracefully degraded)

Mechanisms:

- └ Heartbeat monitoring (detect failures)
 - └ Health checks (replace failed instances)
 - └ Replication (data redundancy)
 - └ Circuit breakers (prevent cascading failures)
-

2.2.3 Redundancy

Duplication of critical components to prevent failures

Database Redundancy:

Primary (Master) → Replicates → Standby (Slave 1)

→ Standby (Slave 2)

→ Standby (Slave 3)

Server Redundancy:

Multiple instances behind load balancer

Network Redundancy:

Multiple internet connections

Power Redundancy:

Backup generators, UPS (uninterruptible power supply)

Active-Active vs Active-Passive

Active-Active (Both processing):

Server 1 → Load Balancer → Processing

Server 2 → Load Balancer → Processing

Failure of Server 1: Server 2 handles 100% load

Better utilization, both working

Active-Passive (Standby):

Server 1 (Active) → Processing

Server 2 (Passive) → Idle (standby)

Failure of Server 1: Server 2 takes over

Wasted resources (Server 2 idle), but simpler

2.3 Consistency

Definition

Consistency ensures all users see the same data at the same time.

Models

1. **Strong Consistency:** All users always see latest data
Write: "Alice's balance = \$100"
All subsequent reads: \$100
Pros: Data always current
Cons: Higher latency (wait for all replicas to update)
2. **Eventual Consistency:** Replicas eventually converge to same state
Write: "Alice's balance = \$100"
Replica 1 (US): \$100 ✓
Replica 2 (EU): Still \$90 (lagging)
Replica 3 (Asia): Still \$90 (lagging)
After 1 second: All replicas show \$100

Pros: Low latency, high availability

Cons: Temporary inconsistency

3. **Causal Consistency**: Causally related operations ordered

Client A: Writes X = 1

Client A: Reads X (sees 1)

Client B: Reads X (sees 1)

But unrelated writes can be inconsistent across replicas

2.4 Performance

2.4.1 Response Time

Time taken to process request and return response

Typical targets (web API):

└─ p50 (median): 50-100ms

└─ p95: 200-500ms

└─ p99: 1000-2000ms (outliers acceptable)

└─ p99.9: < 10000ms

Optimization:

└─ Caching (reduce DB queries)

└─ Async processing (return immediately, process later)

└─ Database optimization (indexes, query optimization)

└─ CDN (serve from nearby location)

2.4.2 Throughput

Requests processed per unit time

Measurement:

Throughput = Requests / Second (RPS)

Examples:

└─ Small API: 100 RPS

└─ Medium API: 1,000 RPS

└─ Large API: 10,000+ RPS

Scaling:

Add more servers horizontally

Request 1 → Server 1

Request 2 → Server 2

Request 3 → Server 3

Throughput increases linearly

2.4.3 Latency

Delay before response starts

Components:

1. Network latency: Time to reach server
2. Processing latency: Server processing time
3. Queuing latency: Wait in queue if server busy

Total Latency = Network + Queuing + Processing

Optimization:

- └─ CDN (reduce network latency)
 - └─ Load balancing (reduce queuing)
 - └─ Async processing (return immediately)
 - └─ Caching (reduce processing time)
-

2.4.4 Concurrency

Ability to handle multiple simultaneous requests

Single-threaded:

Request 1 → Process (100ms) ✓

Request 2 → Wait (queue) → Process (100ms) ✓

Total: 200ms

Multi-threaded (4 threads):

Request 1 → Thread 1 (100ms) ✓

Request 2 → Thread 2 (100ms) ✓

Request 3 → Thread 3 (100ms) ✓

Request 4 → Thread 4 (100ms) ✓

Total: 100ms (parallel)

Considerations:

- └─ Thread pool size: Too small (queuing), too large (overhead)
 - └─ Context switching: More threads = overhead
 - └─ Database connections: Limit concurrent DB connections
 - └─ Memory: Each thread uses memory
-

2.4.5 Load Balancing

See 1.4.2.1 Load Balancing (previously detailed)

2.5 CAP Theorem

Definition

CAP Theorem (Eric Brewer, 2000) states that distributed systems can guarantee at most 2 of 3 properties:

- Consistency
 - Availability
 - Partition Tolerance
-

The Three Properties

1. **Consistency (C):** All nodes see same data
Write X = 1 to Node A
Node A: X = 1 ✓
Node B: X = 1 ✓ (must be updated)
Node C: X = 1 ✓ (must be updated)
Strong guarantee, but requires coordination
2. **Availability (A):** Every request gets response
Any request → Response (success or reasonable error)
System never says "I'm too busy" or "Can't respond"
Must always be responsive
3. **Partition Tolerance (P):** Survives network partitions
Network split: Nodes A,B on one side; C,D on other
No communication between partitions
System continues operating (separate responses)

The Trade-off

In case of network partition, choose C or A (can't have both)

CA (Sacrifice Partition Tolerance):

- Consistent + Available but fails on partitions
- Traditional SQL databases (monolithic)
- All nodes must be connected
- Problem: Network partitions happen!

CP (Sacrifice Availability - Consistency & Partition Tolerance):

- Consistent + Tolerates partitions but may not respond
- When partition happens: Choose consistency over availability
- Return error to maintain consistency
- Example: Banking system (never show inconsistent balance)

AP (Sacrifice Consistency - Availability & Partition Tolerance):

- Available + Tolerates partitions but eventually consistent
- When partition happens: Choose availability over consistency
- Nodes continue responding (may be outdated)
- Example: Social media, e-commerce

Real-world Examples

Google Bigtable: CP

- Consistent across data centers
- Handles partitions
- May not be available during partition

Amazon DynamoDB: AP

- Available (always responds)
- Partitions continue operating
- Eventually consistent (replicas catch up)

Traditional RDBMS: CA

- Consistent (ACID)
- Available (always responsive)
- Fails on partitions (rare, assumed won't happen)

2.6 Cost Optimization

Definition

Cost Optimization reduces infrastructure and operational expenses while maintaining performance.

2.6.1 Resource Optimization

1. Right-sizing: Use appropriate server capacity
 - └ Current: 16 CPU, 64 GB RAM, 20% utilized
 - └ Optimized: 4 CPU, 16 GB RAM, 80% utilized
 - └ Savings: 75% cost reduction
2. Reserved Capacity: Commit long-term, get discount
 - └ On-demand: Pay per hour, no commitment
 - └ Reserved: Commit 1-3 years, 30-70% discount
 - └ If usage stable: Use reserved
3. Spot Instances: Use unused capacity, 70-90% cheaper
 - └ Risk: Can be terminated with 2-minute notice
 - └ Use: Non-critical, fault-tolerant workloads
 - └ Batch jobs, testing environments
4. Autoscaling: Scale up when needed, down when not
 - └ Peak hours: 10 servers
 - └ Off-peak: 2 servers
 - └ Savings: Only pay for what you use
5. Caching: Reduce database load
 - └ Cache hits: No database query (cost-free)
 - └ Cache miss: Database query (pay for it)
 - └ Higher hit rate: Lower costs
6. Multi-tier Storage:
 - └ Hot data: Fast SSD (expensive, small)

- └ Warm data: Standard storage (moderate cost)
 - └ Cold data: Archive (cheap, slow)
7. Regional choice:
- └ Data center choice affects cost
 - └ Some regions cheaper than others
 - └ Consider: Cost vs latency trade-off
 - └ Use: Local region unless latency critical
-

2.7 Data Consistency

Already covered in 2.3 Consistency - ensuring data integrity across replicas

2.8 Single Point of Failures

Already covered in 1.9 - SPOF (Single Point of Failure)

SUMMARY

This comprehensive roadmap covers all major system design concepts:

Functional Requirements enable core system capabilities:

- CDN distributes content globally
- Databases (SQL/NoSQL) store data
- APIs enable service communication
- Caching improves performance
- Security protects data
- Monitoring/Logging provides visibility

Non-Functional Requirements ensure system quality:

- Scalability handles growth
- Availability ensures reliability
- Consistency maintains data integrity
- Performance meets user expectations
- CAP theorem guides architecture decisions
- Cost optimization reduces expenses

Key Takeaways for Interviews:

1. Understand trade-offs (consistency vs availability, latency vs throughput)
 2. Know when to use which technology (Kafka vs RabbitMQ, Redis vs Memcached)
 3. Design for failure (redundancy, health checks, circuit breakers)
 4. Scale horizontally when possible
 5. Cache aggressively, monitor constantly
 6. Document assumptions and trade-offs clearly
-

Good luck with your system design interviews!

